机器人避障寻径实验报告

姓名：郎子萱

学号：2452086

学院：计算机科学与技术学院

专业：数据科学与大数据技术

1.基础算法： A* 算法

1.1 A* 算法原理

A* (A-Star) 算法是一种在静态路网中求解最短路径极其有效的**启发式搜索算法**。它结合了 Dijkstra 算法（保证能找到最短路径）和贪婪最佳优先搜索算法（极大地提高搜索效率，具有方向性）的优点。

A* 算法的核心在于其节点状态评估函数：

$$f(n) = g(n) + h(n)$$

- $g(n)$：从起始点 S 沿着已生成的路径到达当前考察节点 n 的实际代价。在本实验栅格地图中，机器人每次可以向“上下左右”四个方向移动一步，每走一步的代价设为 1。
- $h(n)$：启发函数，表示从节点 n 到目标终点 E 的估计最小代价。考虑到本场景中机器人只能沿水平或垂直方向移动（不支持对角线移动），因此启发函数采用**曼哈顿距离** (Manhattan Distance) 最为合适。形式为：$h(p_1, p_2) = |x_1 - x_2| + |y_1 - y_2|$。
- $f(n)$：从起始点经过节点 n 到达终点的综合总代价估计。每次进行搜索扩展时，算法总是优先挑选 $f(n)$ 最小的节点进行探索。

算法逻辑流程如下：

1. **初始化**：创建一个优先队列 `open_list` （用于存放待考察且已知代价的节点），并将起点 S 放入。创建一个集合 `visited` 用于记录已探索过的节点，并使用 `came_from` 字典记录路径上的父子节点关系（用于到达终点后回溯完整路径）。
2. **取出最优节点**：进入循环。每次从 `open_list` 中取出评估值 $f(n)$ 最小的节点作为当前节点 $current$。
3. **终止条件**：如果 $current$ 即为终点 E，说明已经找到了最短路径。通过 `came_from` 字典不断寻找父节点逆推，生成完整的最短路径序列并结束搜索。
4. **节点展开**：如果 $current$ 尚未到达终点，将其加入 `visited` 。遍历其周围相邻（上、下、左、右）且位于地图边界内的非障碍物邻居节点。如果计算出到达该邻居的代价 $g(neighbor)$ 比目前记录的更优（或该邻居还未被探索），则更新该邻居的 g 代价和评估值 f，将其父节点指向 $current$，然后将其推入 `open_list` 队列中等待后续展开。重复该过程直到找到终点。

1.2 算法实现

第一部分：启发函数与初始化阶段

这个地图是四向有限移动，所以使用曼哈顿距离作启发函数估计最低代价。然诺初始化 `open_list` 优先队列，存入格式为 `(f_score, g_score, (row, col))` 的数据，保证能够依 $f(n)$ 大小自动筛选优先扩展的节点。另外创立字典和集合分别记录父节点和走过的记录。

```python
import heapq

def heuristic(a, b):
    # 启发函数：曼哈顿距离 (Manhattan distance)
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def a_star_search(grid, start, end):
    rows, cols = grid.shape
    # 四个移动方向：上下左右 (每步代价为1)
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    # 优先队列 open_list，保存记录待扩展状态
    open_list = []
    heapq.heappush(open_list, (0 + heuristic(start, end), 0, start))

    came_from = {}       # 记录节点的父节点，用于最终的回溯路径
    g_score = {start: 0} # 记录起点到当前点的已知实际代价 g(n)
    visited = set()      # 用于剔除已经完全遍历过的点(闭表)

    explored_nodes = []  # 记录所有被访问过的点，供后续可视化展示使用
```

第二部分：主循环与目标测试阶段

进入主循环，每次不断地通过 `heapq.heappop` 提取当前评价指标 f 值最小的节点进行尝试。优先记录其位置用于过程可视化，然后立即进行终点核验。如果它是终点，则利用预先保存的 `came_from` 路径记忆，经追溯、反转后即可返回完整路径。如果不是终点，则打上 `visited` 标签。

```python
while open_list:
    # 取出代价最小（即 f 值最小）的节点
    _, current_g, current = heapq.heappop(open_list)

    # 记录已探索点
    if current not in explored_nodes:
        explored_nodes.append(current)

    # 触达终点：按找寻历史回溯完整路径并返回
    if current == end:
        path = []
        curr = current
        while curr in came_from:
            path.append(curr)
            curr = came_from[curr]
        path.append(start)
        path.reverse() # 倒溯所以需要反转成正向
        return path, explored_nodes

    # 如果已被完全扩展即抛弃跳过
    if current in visited:
        continue
    visited.add(current)
```

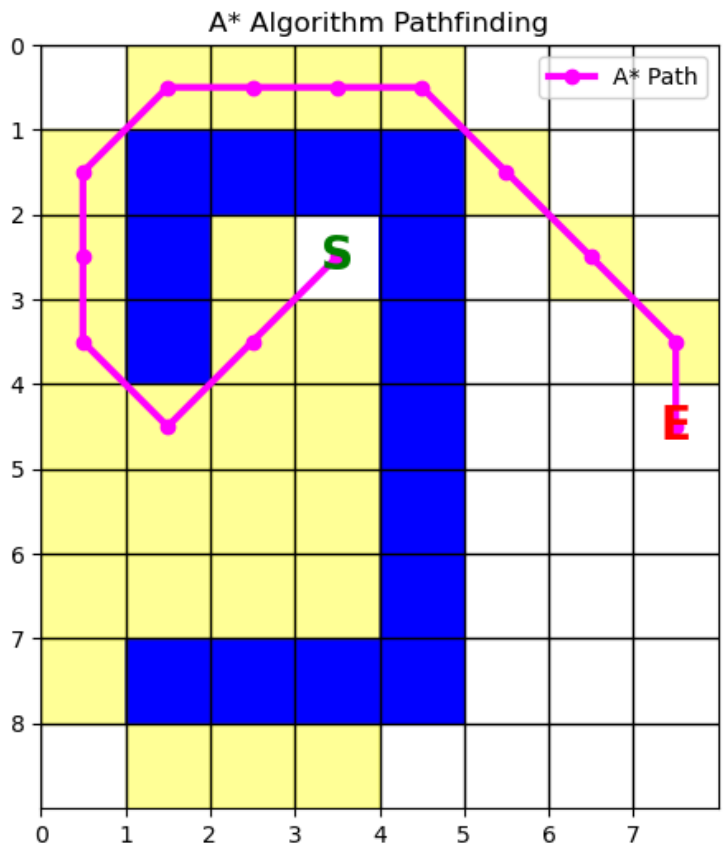
第三部分：节点展开与状态更新阶段

对于尚未到达的目标，算法依次扩散周围的 4 个邻居方向。通过地图维度（行与列）做越界阻断，遇到 1 即说明是防区障碍物，同样阻断。排除无效点后，计算去该邻近点的假设代价 `tentative_g` ，如果判定是一条更快捷的路径，或者是个陌生的新节点，就更新其各项开销、绑定属于它的父节点，计算综合价值参数 f 送入到下一回合的优先队列进行比拼。

```
        # 遍历四个方向的邻居
        for d in directions:
            neighbor = (current[0] + d[0], current[1] + d[1])

            # 边界合法性检查
            if 0 <= neighbor[0] < rows and 0 <= neighbor[1] < cols:
                # 避障检查 (grid值为1处为障碍物)
                if grid[neighbor[0], neighbor[1]] == 1:
                    continue

                tentative_g = current_g + 1

                # 发现更短的路径或开启了新节点，则更新记录状态
                if neighbor not in g_score or tentative_g < g_score[neighbor]:
                    came_from[neighbor] = current
                    g_score[neighbor] = tentative_g
                    f = tentative_g + heuristic(neighbor, end) # f(n) = g(n) + h(n)
                    heapq.heappush(open_list, (f, tentative_g, neighbor))

    # 没有找到通往终点的路径时返空集
    return None, explored_nodes
```

1.3 运行结果与可视化

最后A*算出来的路径如下，最短步数是13步，确实是最短的：



2.基础算法： BFS 算法

2.1 BFS 算法原理

广度优先搜索 (Breadth-First Search, BFS) 是一种经典的图搜索“盲目搜索”算法。其核心思想是从起点出发，以“水波纹”般向四周一圈一圈地扩散，依次遍历所有距离起点最近的节点。

由于本栅格地图场景中上下左右移动步数代价均为 1，且 BFS 的特性保证了每一次波及到的总是距离起点步数最少的外层节点，因此只要搜索触碰到终点，该条路径就必定是所有解中的最短路径。而在开销和时间方面，BFS 必须遍历大量与终点背道而驰的节点空间，其盲目性使其探索成本大幅高于具备启发导向（曼哈顿距离）的 A* 算法。

2.2 算法实现

与 A* 不同，BFS 不需要评估总价 $f(n)$，只需采用**先进先出 (FIFO)** 的数据结构 `deque` (双端队列) 即可保证广度优先的层级扩散特性。以下为 `bfs.py` 中逻辑的详细拆解：

第一部分：队列与集合初始化阶段
导入队列工具 `deque` ，设定起点 `start` ，由于无需进行优先级排序，不需要 `heapq` 。而是将起点压入队列，并存入防重复的 `visited` (闭表) 集合中，设定路径追踪字典 `came_from` 。

```python
from collections import deque

def bfs_search(grid, start, end):
    rows, cols = grid.shape
    # 四个移动方向：上下左右
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    # 使用双端队列 deque 实现先进先出 (FIFO) 的结构
    queue = deque([start])

    # 用集合记录已经访问过的节点，防止重复访问
    visited = set([start])

    came_from = {start: None} # 用于记录前驱节点，方便找到目标后回溯路径
    explored_nodes = []       # 用于记录探索顺序，供后续可视化展示
```

第二部分：主循环出队与终点校验阶段

不停从队首 `popleft()` 拔出当前最早入队的节点进行访问校验。此时节点如果是终点，由于广搜的层级拓扑性质，这一支线必定是最短路。随即执行与 A* 一致的 `came_from` 路径反抽重组操作。

```python
    while queue:
        current = queue.popleft() # 弹出最早进入队列的层级节点

        # 记录已探索点
        if current not in explored_nodes:
            explored_nodes.append(current)

        # 如果当前节点就是终点，回溯出完整路径
        if current == end:
            path = []
            curr = current
            while curr is not None:
                path.append(curr)
                curr = came_from[curr]
            path.reverse()
            return path, explored_nodes
```

第三部分：四向拓展与访问注册阶段

同样是进行四向位移并剥离地图越界和障碍物。与 A* 的松弛更新不同的是：BFS 对于新搜到的可用点，立马直接加盖已访问章 `visited.add()` 以免其它支路重新踏入，并推入队尾 `queue.append(neighbor)` 等待下一层级的调遣。

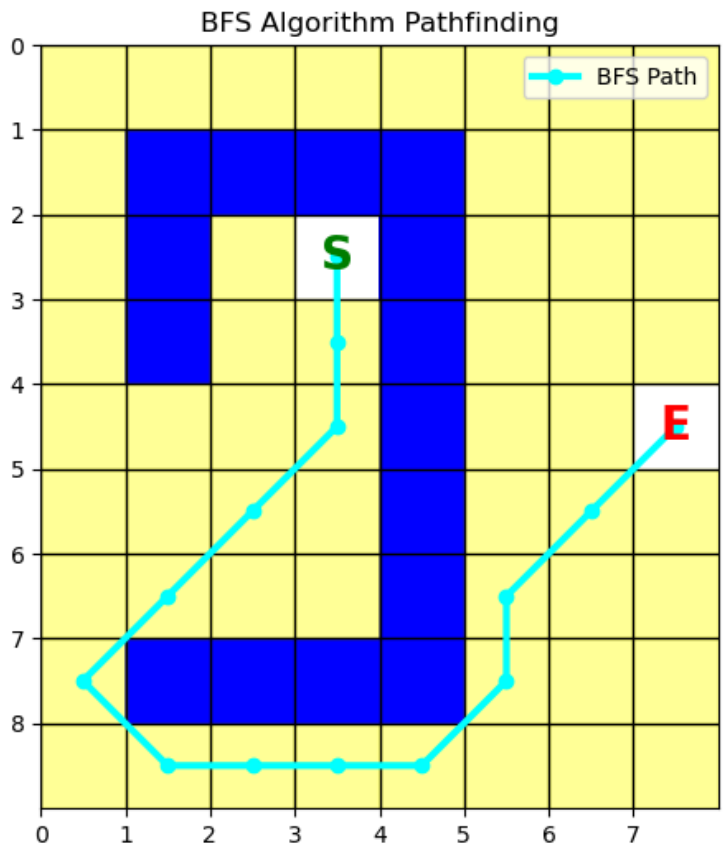
```
        # 遍历四个方向的邻居
        for d in directions:
            neighbor = (current[0] + d[0], current[1] + d[1])

            # 边界检查
            if 0 <= neighbor[0] < rows and 0 <= neighbor[1] < cols:
                # 避障检查与重访检查：值为1处为障碍，不可访问
                if grid[neighbor[0], neighbor[1]] == 0 and neighbor not in visited:
                    visited.add(neighbor)
                    came_from[neighbor] = current
                    queue.append(neighbor) # 队尾入列，等待下一圈扩散

    # 若遍历完整个连通区域仍找不到终点，则返回 None
    return None, explored_nodes
```

2.3 运行结果与可视化

BFS成功给出了一条 13 步长的有效最短路径（具体坐标走向由于入队先后判定与A*有区别区别，但总长同为最优）。

- **BFS 寻路成功步数**：13 步
- **BFS 探索总节点数**：57 个，呈大范围盲目水波纹铺列（相比之下，A*由于有终点拉力，探索点远少于此）。

BFS 算法执行结果及宽域探索空间可视化如下：



3 其他解决方案：Dijkstra 算法

这里对应第(4)小题，考虑报告的逻辑流畅性，放到了和A*、BFS同一部分

3.1 Dijkstra 算法原理与逻辑

Dijkstra 算法是一种经典的**单源最短路径** (Single-Source Shortest Path) 算法，由荷兰计算机科学家 Edsger W. Dijkstra 在 1956 年提出。相比于 A* 算法，Dijkstra 的特点是**不使用任何启发函数**，而是仅基于从起点到各节点的实际距离进行最优决策。

Dijkstra 与 A 的本质差异：

- **A* 算法**：使用 $f(n) = g(n) + h(n)$，其中 $h(n)$ 是关于到目标的启发估计。通过提供目标方向的拉力，A* 能够提早发现终点并减少探索范围。
- **Dijkstra 算法**：仅使用 $f(n) = g(n)$，即从起点到当前节点的实际距离。没有启发函数的指导，Dijkstra 会向所有方向均匀地扩散，形成"水波纹"式的探索模式，但完全避免了选择不当启发函数导致的偏差问题。

Dijkstra 的主要特性：

1. **无启发函数**：不需要设计 $h(n)$，避免启发函数误导的风险。
2. **完全正确性**：在所有边权非负的图中保证找到最短路径。

3. **探索模式**：以起点为中心的圆形波状扩散，探索范围通常比 A* 大。
4. **时间复杂度**：使用最小堆优化时为 $O((V+E)\log V)$，其中 V 为节点数，E 为边数。

3.2 算法详细实现

第一部分：初始化与数据结构

初始化距离字典记录从起点到各节点的最短距离（初始为无穷），并使用最小堆优先队列按距离值排序。无需计算启发函数，直接以实际距离作为优先级。

```
def dijkstra_search(grid, start, end):
    rows, cols = grid.shape
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1), (-1, -1), (-1, 1), (1, -1), (1, 1)]

    # 初始化距离表：记录起点到各点的最短距离
    distance = {start: 0}

    # 优先队列：(当前距离, 节点坐标) - 只含距离，无启发值
    pq = [(0, start)]

    # 已访问集合（闭表）
    visited = set()

    # 记录父节点，用于回溯路径
    came_from = {}
    explored_nodes = []
```

第二部分：主循环与松弛操作

进入主循环，每次从优先队列中取出距离最小的节点。对其邻居进行**松弛操作** (Relaxation)：计算经过当前节点到达邻居的距离，如果比之前记录的距离更短，则更新邻居的距离值和父节点记录，并将邻居压入优先队列。

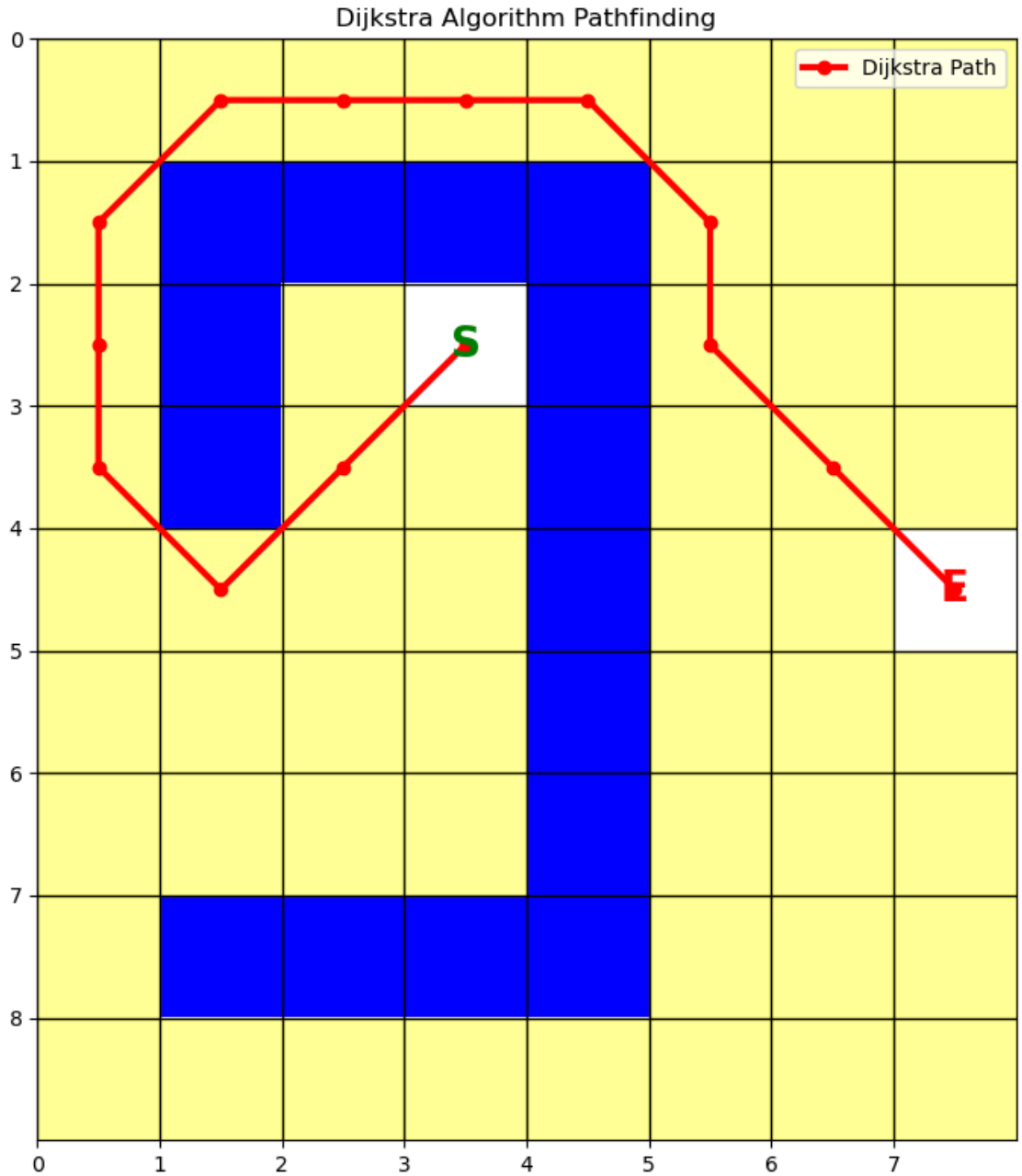
```python
while pq:
    current_dist, current = heapq.heappop(pq)

    # 记录探索过的点
    if current not in explored_nodes:
        explored_nodes.append(current)

    # 如果已访问过，跳过
    if current in visited:
        continue

    visited.add(current)

    # 到达终点，立刻回溯并返回
    if current == end:
        path = []
        curr = current
        while curr in came_from:
            path.append(curr)
            curr = came_from[curr]
        path.append(start)
        path.reverse()
        return path, explored_nodes

    # 松弛邻居节点
    for d in directions:
        neighbor = (current[0] + d[0], current[1] + d[1])

        # 边界和避障检查
        if not (0 <= neighbor[0] < rows and 0 <= neighbor[1] < cols):
            continue
        if grid[neighbor[0], neighbor[1]] == 1:
            continue
        if neighbor in visited:
            continue

        # 计算移动代价：对角线 1.414，直线 1.0
        move_cost = 1.414 if (d[0] != 0 and d[1] != 0) else 1.0
        new_dist = current_dist + move_cost

        # 如果找到了更短的路径，则更新
        if neighbor not in distance or new_dist < distance[neighbor]:
            distance[neighbor] = new_dist
            came_from[neighbor] = current
            heapq.heappush(pq, (new_dist, neighbor))
```

3.3 Dijkstra 与 A*/BFS 的对比运行结果

在同一个 8×8 的基础地图（`map_config.py`）上运行 Dijkstra 算法：

- **Dijkstra 寻路步数：13 步**（与 A* 和 BFS 一致，均为最短）
- **Dijkstra 探索总节点数：57 个**（与 BFS 完全相同，反映出两者都是无启发的扩散式搜索）
- **特点**：由于没有启发函数指向终点，Dijkstra 会向四周均匀扩散，探索模式呈圆形波纹，因此探索节点数与 BFS 持平，都显著多于 A*（A* 仅需 23 个探索节点）。

Dijkstra 算法执行结果可视化如下：

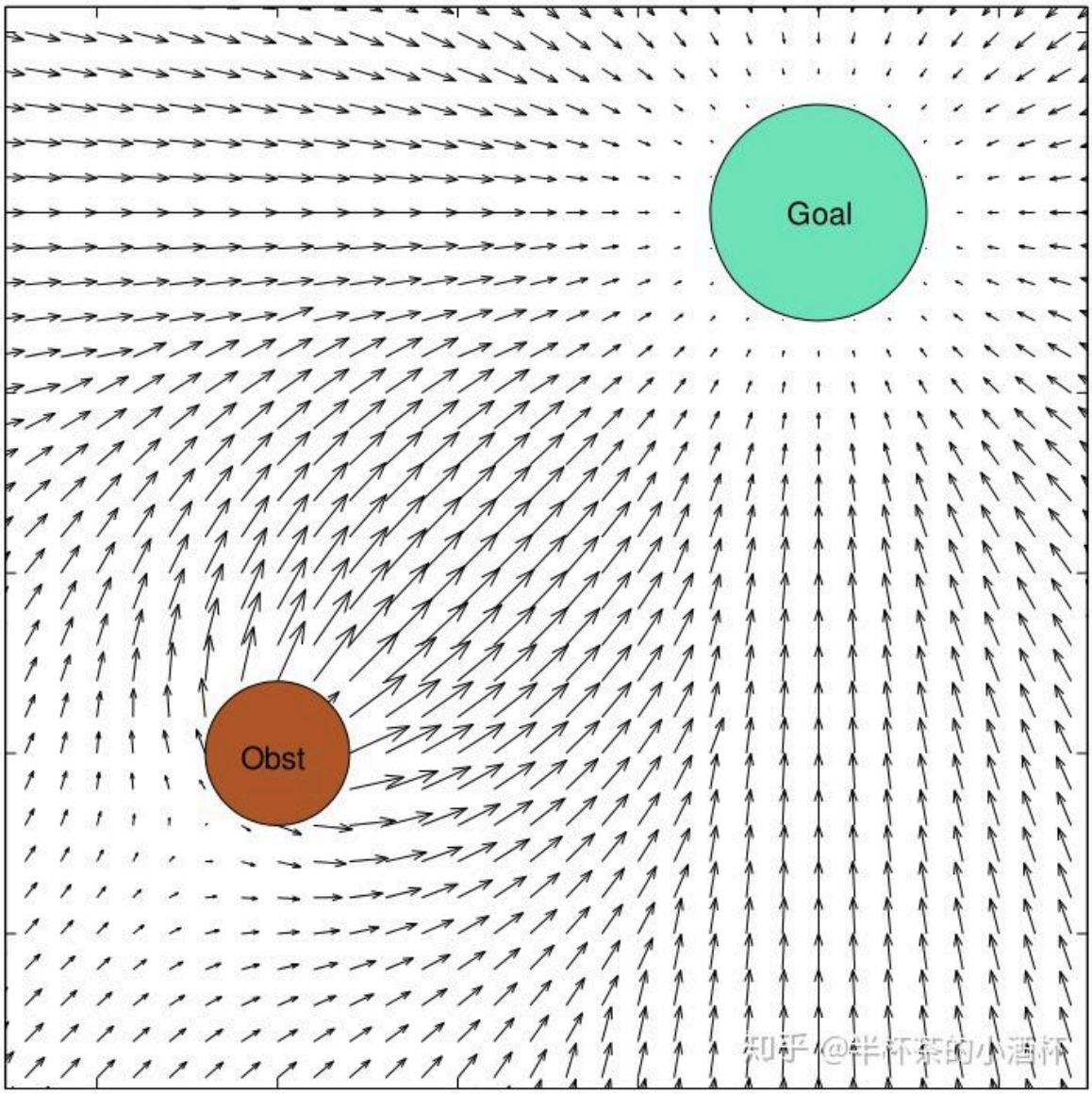


4. 进阶算法：人工势场法 (APF) 与演进

4.0 APF方法的介绍

(通俗而详细的讲解可以参考https://zhuanlan.zhihu.com/p/144816424，part4.0的内容摘自这篇专栏)

人工势场法(APF)是一种经典的机器人路径规划算法。该算法将目标和障碍物分别看做对机器人有引力和斥力的物体，机器人沿引力与斥力的合力来进行运动。



人工势场包含两种两种力场:运动目标位置所形成的引力场(Attractive Field)和障碍物所形成的斥力场(Repulsive Field)。

$$U(q) = U_{att}(q) + U_{rep}(q)$$

其中，
$U_{att}(q)$是引力场，引导机器人向目标位置运动；
$U_{req}(q)$是斥力场，引导机器人避开障碍物。

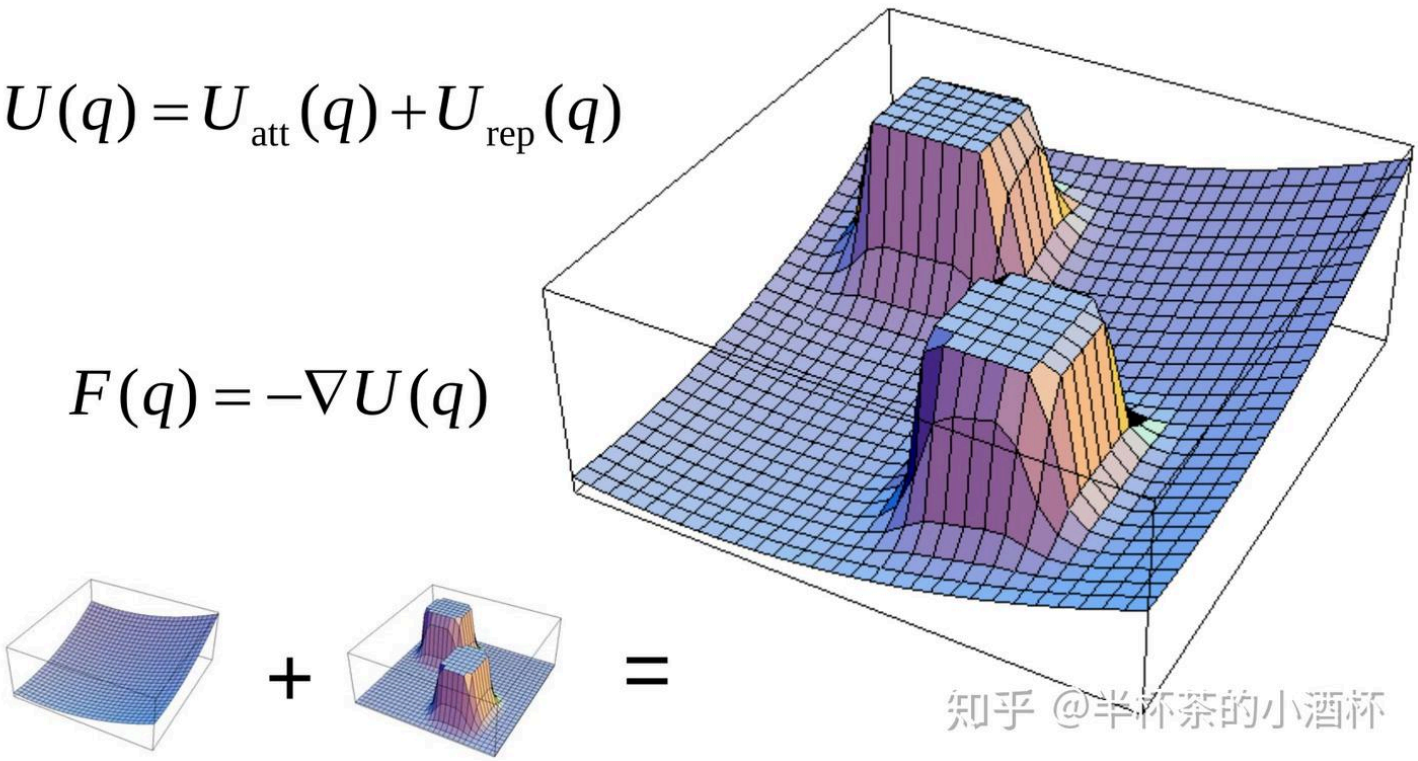
最常见的引力场函数(Attractive Field)如下：

$$U_{att}(q) = \begin{cases} \frac{1}{2}\zeta d^2(q, q_{goal}) & d(q, q_{goal}) \le d^*_{goal} \\ d^*_{goal}\zeta d(q, q_{goal}) - \frac{1}{2}\zeta (d^*_{goal})^2 & d(q, q_{goal}) > d^*_{goal} \end{cases}$$

斥力场常用函数(Repulsive Potential)如下：

$$U_{rep}(q) = \begin{cases} \frac{1}{2}\eta\left(\frac{1}{D(q)} - \frac{1}{Q^*}\right)^2, & D(q) \le Q^* \\ 0, & D(q) > Q^* \end{cases}$$

将引力场和斥力场叠加，就形成了人工势力场。



4.1 Basic APF 及局部极小值问题

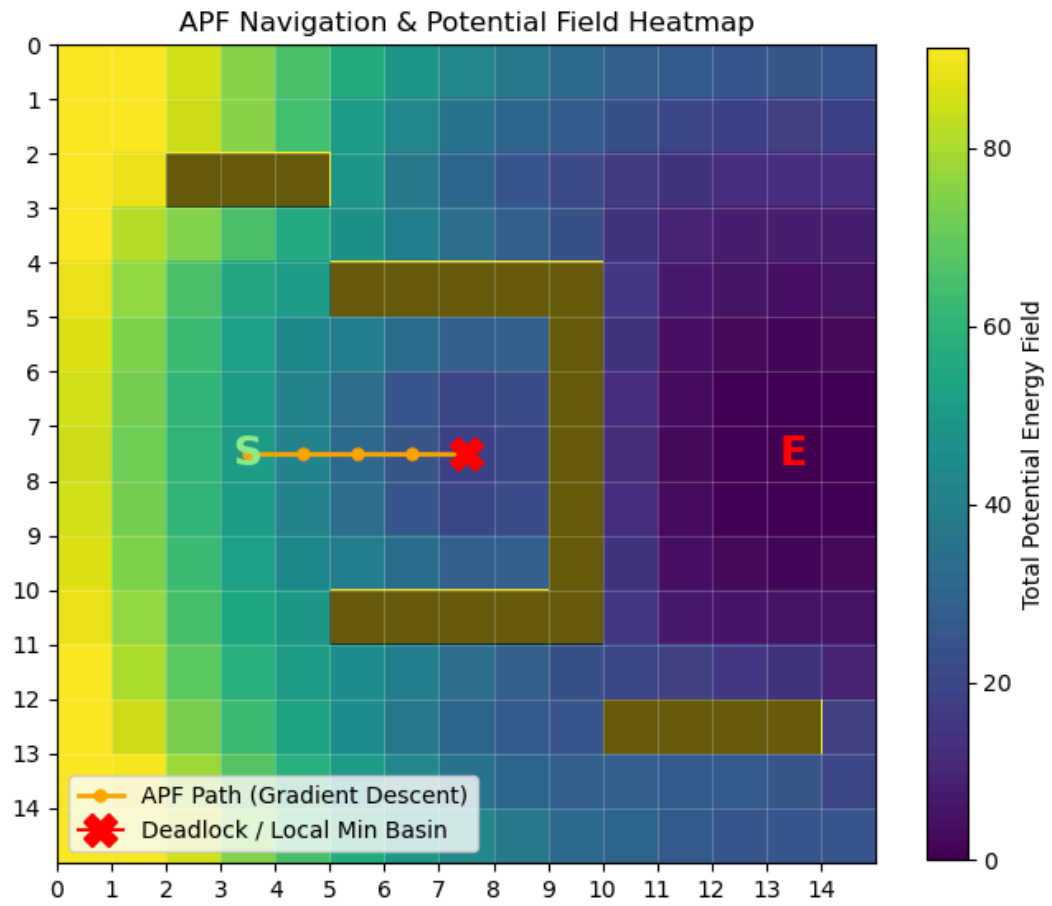
在进阶部分，为APF构建了一个包含 U 型陷阱的复杂 15x15 栅格地图，并尝试使用传统的人工势场法进行控制。人工势场法（APF）通过向目标点施加虚拟的引力，向障碍物施加虚拟的斥力，引导机器人如同物理质点顺着势能梯度下降运动。
但是这种传统势场有天生缺陷：很容易陷入**局部极小值**。在 U 型陷阱内部，向前的引力可能会被前方障碍物的斥力完全抵消或处于低谷盆地，导致势能全方位呈上升趋势，机器人被困死。

以下是传统 APF 势能计算函数的代码实现：

```python
def calculate_potential(pos, end, obs, k_att=1.0, k_rep=20.0, rho_0=3.0):
    # 1. 目标吸引力势能：距离越远引力越大 (抛物线型)
    d_goal = np.sqrt((pos[0] - end[0])**2 + (pos[1] - end[1])**2)
    u_att = 0.5 * k_att * (d_goal ** 2)

    u_rep = 0.0
    # 2. 障碍物排斥力势能：在影响半径 rho_0 内，距离越近斥力以反比例激增
    for ob in obs:
        d_obs = np.sqrt((pos[0] - ob[0])**2 + (pos[1] - ob[1])**2)
        if d_obs > 0 and d_obs <= rho_0:
            u_rep += 0.5 * k_rep * ((1.0 / d_obs - 1.0 / rho_0) ** 2)

    return u_att + u_rep
```

运行测试时，这套纯基于上述函数的局部势场贪心搜索法在坐标一头扎进了 U 型陷阱的中央并报错卡死。即使环顾四周也无法找到能够继续梯度下降的方向。
这套死锁逻辑的探路主循环呈现如下特征：总是优先挑选更低势能方向，一旦无法下降直接判定为死锁报错：

```python
        found_better = False
        for d in directions:
            nr, nc = current[0] + d[0], current[1] + d[1]
            if 0 <= nr < grid.shape[0] and 0 <= nc < grid.shape[1] and grid[nr, nc] == 0:
                neighbor = (nr, nc)
                u = calculate_potential(neighbor, end, obs)

                # 如果存在比当前所处位置更低的势能，才认为可以前进
                if u < min_u:
                    min_u = u
                    best_next = neighbor
                    found_better = True

        # 贪心探索失败：无法下降，陷入死锁
        if not found_better:
            print(f">>> 陷入局部极小值 (Deadlock) 在坐标: {current}! 无法继续前进。")
            return path, explored, False
```

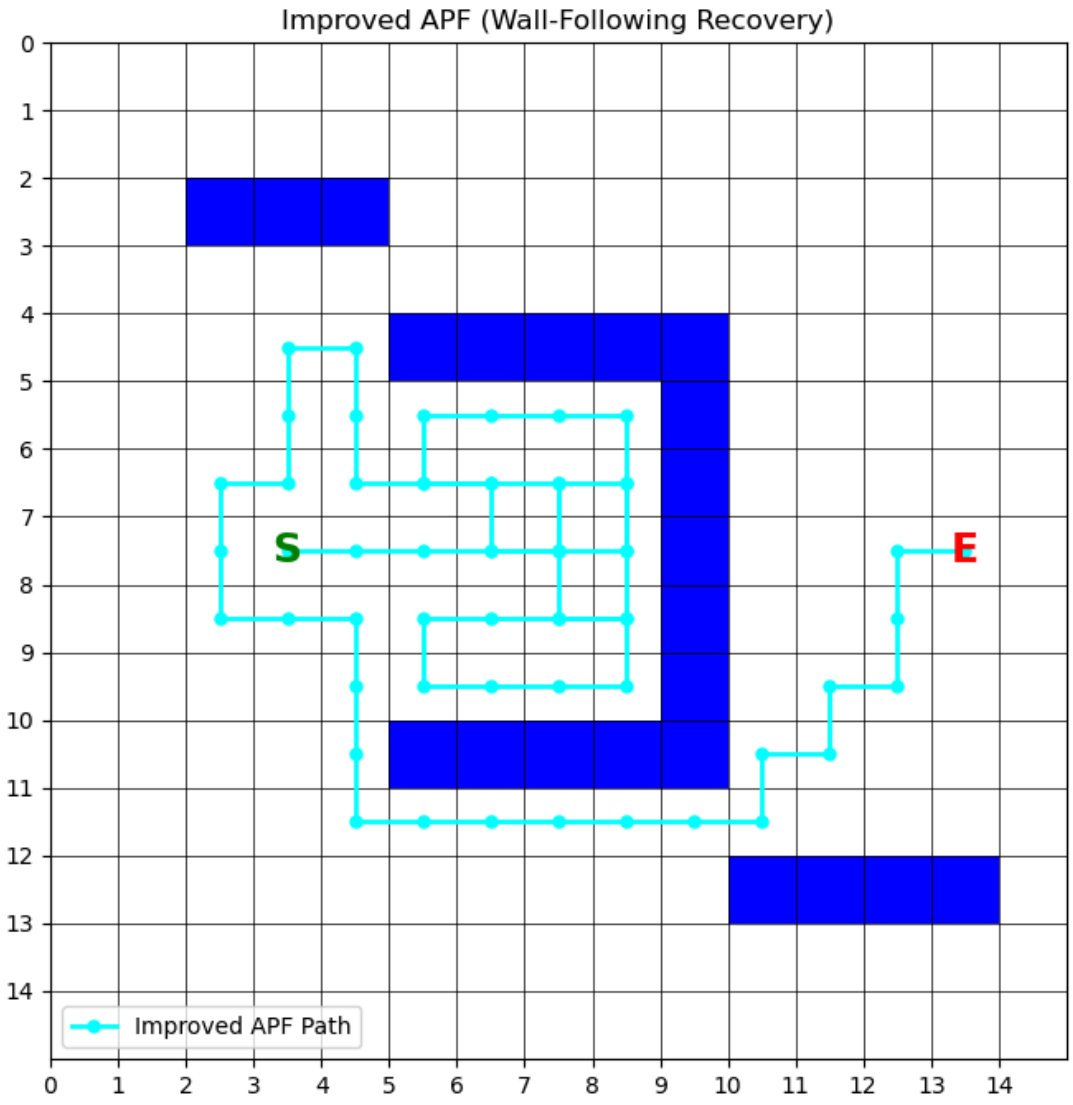


4.2 改进方向一（失败）：引入动态记忆惩罚脱困

为应对局部极小值，需要让算法具备**自救和脱困**能力，思路是：如果机器人掉入势能坑，就不断往这个“坑”里投掷沙土（强行抬高反复踩踏位置的势能）。通过记录每个节点的访问次数，并给予巨额惩罚参数 `* 100.0` ，**使得原先走投无路的低谷在势能场视觉上变成高山**。这迫使机器人顺着被垫高的地形产生出走坑外的漫游倾向。
以下为携带了“填坑记忆”的势力叠加评判尝试逻辑：

```python
    visited_counts = {}

    for _ in range(max_iter):
        if current == end:
            return path, explored, True

        # “踩踏”记录：当前位置经过次数叠加
        visited_counts[current] = visited_counts.get(current, 0) + 1

        best_next = current
        min_u = float('inf')

        neighbors = get_neighbors(current, grid)
        # 用带惩罚的合成势能去探查前路并比较出“相对势能最小”的方向
        for neighbor in neighbors:
            # 获取原客观存在的物理场势能
            u = calculate_potential(neighbor, end, obs)

            # 引入巨额的动态记忆惩罚：越是踩过的旧路，人造势能越剧烈升高
            u += visited_counts.get(neighbor, 0) * 100.0

            if u < min_u:
                min_u = u
                best_next = neighbor
```

虽然这一改进方法确实能够通过极其暴力且盲目的方式打破了死锁，并最终历经数千次迭代碰撞到达了终点，但是因为它带有“填水效应”，机器人**本质上只是在坑内一圈圈漫无目的地瞎撞反复抬高惩罚**，直至“水漫金山”后溢出死路。这样的运行轨迹严重偏离了最短或平滑路径范畴，性能低下。



4.3 改进方向二（成功）：混合启发式搜索，APF 结合局部 A* 脱困

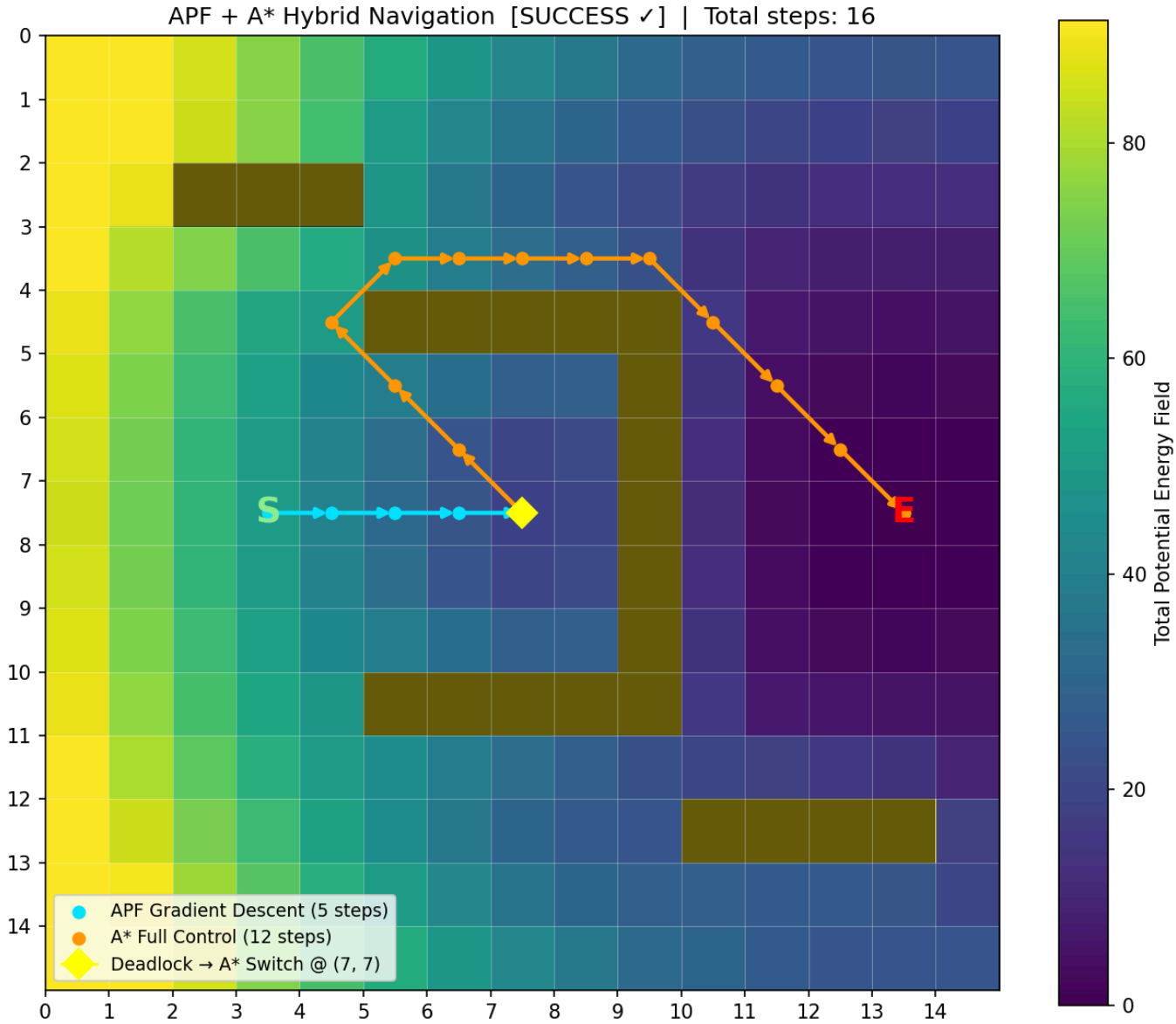
鉴于盲目的记忆惩罚法脱困效率极低且路径杂乱，进行了第二版逻辑优化尝试（`apf_improved(v2).py`）：**将局部贪心与全局启发式算法硬切换结合**。

核心优化逻辑分为两个阶段（Phase）：前期运用传统 APF 算力消耗极小、响应极快的特点，让机器人顺着势能平滑快速下降；而一旦检测到 `found_better == False` ，就说明撞上了 U 型坑底或是障碍物形成的死锁，此时立刻抛弃 APF 的短视贪心策略，直接原地呼叫 *A 算法全局接管*。A* 算法会从当前的死锁位置计算出一条直通终点的无碰撞最短路径，随后机器人将逐一跟随 A* 规划的队列直接走出困境，不再受原本环境斥力的干扰。这种“混合动力”既兼顾了无障碍区的快速平滑移动，也解决了了人工势场法遇到复杂地形容易挂起的问题。

混合算法中的侦测切换核心代码如下：

```python
# ── Deadlock 侦测 → 硬切换到 A* ────────
if not found_better:
    print(f">>> [步{iteration}] APF deadlock @ {current}, "
          f"切换到 A* 完全接管模式...")

    # 使用 A* 从当前卡死点寻路到终点
    astar_path = astar(grid, current, end)
    if astar_path is None:
        print(">>> A* 也找不到路径，地图无解。")
        return path, explored, False, mode_log

    # 去除当前点，截去首位避免重复，将剩余路径压入接管队列
    astar_queue = astar_path[1:]
    phase       = 'A*'  # 状态机硬切换：后续遍历直接跟 A* 走
    continue    # 下一轮直接走 A* 分支
```

APF+A* 成功给出了一条 16 步长的有效最短路径:可视化如下：


APF + A* Hybrid Navigation [SUCCESS ✓] | Total steps: 16
S
E
APF Gradient Descent (5 steps)
A* Full Control (12 steps)
Deadlock → A* Switch @ (7, 7)
Total Potential Energy Field
0
20
40
60
80

5. 各算法对比分析与总结

5.1 算法性能对比表

算法名称	路径最优性	计算复杂度	内存占用	局部极小值问题	运行速度	适用场景
A*	⭐⭐⭐⭐⭐ 保证最短	O(n log n) 启发式	中等 O(n)	无	取决于h	已知全地图的静态路网寻路 精确导航任务
BFS	⭐⭐⭐⭐⭐ 保证最短	O(n) 完全展开	高 O(n)	无	盲目扩展	栅格小地图 等权边图
Dijkstra	⭐⭐⭐⭐⭐ 保证最短	O((V+E) log V) 堆优化	中等 O(n)	无	均向扩展	加权图/动态权重 标准最短路问题
Basic APF	⭐⭐ 可能被困	O(1) 纯势能计算	极低 O(1)	⚠️ 严重 贪心死锁	极快	无障碍/简单环境 实时反应式控制
APF+记忆惩罚	⭐⭐ 能到达目标	O(n·k) k= 迭代次数	中等 O(n)	✅ 可脱困 但效率极低	数千次迭代	学术验证 极端复杂陷阱
APF+A*混合(v2)	⭐⭐⭐⭐⭐ 保证最短	O(n log n) 混合	中等 O(n)	无 遇困立刻A*	平滑+快速	🏆 **综合最优** 复杂未知环境 实时导航系统

5.2 详细分析

路径最优性： A*、BFS、Dijkstra 和 APF+A*混合都保证找到最短路径；Basic APF 会陷入局部极小值困境；记忆惩罚法虽能脱困但轨迹冗长。其中 Dijkstra 因为没有启发函数，在最坏情况下需要探索更多节点，但其正确性无需依赖启发函数的设计。

计算复杂度： Basic APF 由于只做势能计算最轻量化，但无法处理复杂环境；A* 的启发式搜索通常比 Dijkstra 和 BFS 更高效；Dijkstra 使用堆优化后为 $O((V+E)\log V)$，在稠密图中与 A* 相当；BFS 的理论复杂度为 $O(n)$ 但在栅格地图中需要遍历所有可达节点。混合方案在两者间取得平衡。

内存占用： Basic APF 内存极低（只需存储当前位置和势能值）；其他搜索算法（A*、BFS、Dijkstra）都需维护访问集合、优先队列等，占用 O(n) 内存。

局部极小值处理： Basic APF 直接失败；记忆惩罚虽可脱困但极其低效（可能需数千步）；**APF+A*混合一旦检测到死锁立刻调用全局规划算法，既保证可达性，又保持高效率**。而 Dijkstra、BFS 等无启发搜索天生避免了局部极小值问题。

运行速度与启发效果： Basic APF 极快但不可靠；*A 最快**，因其具有目标导向的启发函数，能提早发现终点；**Dijkstra 次之**，采用均向圆形扩散，探索范围通常大于 A*；**BFS 和 Dijkstra 探索节点数相同**（本实验中都是 57 个），但都远多于 A*（23 个）；混合方案在大多数开放区域快速响应（APF），在困难区域精确规划（A*），是速度与可靠性的完美结合。

5.3 结论与推荐

在本次实验中，*APF+A 混合算法（v2）** 展现了综合优势，特别是在处理包含 U 型陷阱等复杂障碍结构的环境中：

1. **快速平滑导航**：利用人工势场法在开放区域的高效性，使机器人顺势能梯度优雅地接近目标。
2. **智能困境救援**：一旦陷入死锁，立刻转向全局最优规划，彻底规避了局部极小值的陷阱。
3. **保证完整性和最优性**：通过状态机的硬切换，既确保路径必定可达，又保持全局最优的特性。
4. **实际应用价值**：该策略广泛应用于自主移动机器人、无人车辆和无人机的混合导航控制系统中。

关于传统搜索算法的对标： 在纯搜索方案中，A* 凭借启发函数的导向性表现最优，探索节点数仅为 23 个；而 Dijkstra 和 BFS 作为无启发的均向搜索，虽然都能保证找到最短路径，但探索节点数均为 57 个，接近盲目搜索。在动态加权或对手问题场景中，Dijkstra 的无启发特性使其更加可靠（不受启发函数误导），但代价是计算开销。

对比之下，单一的 Basic APF 尽管反应快速，但遇到复杂地形即刻失效；纯粹的无启发搜索（BFS/Dijkstra）虽然保证最优但计算量较大、需要探索大量节点；有启发搜索（A*）虽然高效但依赖启发函数的设计质量；而**APF+A*混合方案通过巧妙的策略选择，实现了"因地制宜、扬长避短"的完美导航**。